

CodeTrackr — Machine Learning Integration Plan

Status: designed and scoped, not built. First written 2026-09-16; redesigned 2026-09-17. Nothing described below exists in the code yet. The project changes this plan needs are listed file by file in `docs/ML_INTEGRATION_CHANGES.md`, and `docs/ML_INTEGRATION_GUIDE.md` explains all of it in simple language with worked examples.

1. The idea in one paragraph

A machine-learning model answers one question about every coding session: **what kind of coding was this?** — *DSA practice, project building, debugging, learning, or setup*. Simple, visible rules then turn those answers into a **persona** for each person ("DSA Warrior", "Project Grinder", "Debugging King") and into **weekly titles inside each group** ("DSA Warrior of CSE-Gang"). The model learns from answers people confirm with **one tap**, so its accuracy can be measured, and it only replaces the hand-written rules if it measurably beats them.

2. Where we honestly are today

Measured on the live database on 2026-09-16:

What we checked	Count	What it means
Sessions with detailed data	4	nowhere near enough to learn from
People sending data	1	a model would only learn one person's habits
New data since 12 September	0	collection has stalled
Completed goals	0	no "right answers" of any kind yet

So no model can be trained yet. The plan is built around that: the first two phases ship useful features **without** machine learning, and those features are what collect the data the model needs.

3. The test machine learning has to pass

We only use ML where all five answers are "yes":

1. **Does it produce something users care about?**
2. **Would simple rules do it badly?** If rules do it well, ML is decoration.
3. **Can we check it is correct?** No way to measure accuracy means no way to claim it works.
4. **Is a wrong answer harmless?** These are friends competing in a group.
5. **Can we explain the output?** Every number in CodeTrackr is explainable; the model must be too.

4. Ideas considered and rejected

Idea	Verdict	Why
Skill level (beginner / moderate / advanced)	Rejected	Activity is not skill: a beginner can type 500 lines a day while an expert reads more and deletes more. There is no answer key, so accuracy cannot be measured. A public "Beginner" tag hurts in a friends' competition, and typing more would game it.
Personas straight from k-means	Rejected as the main design	k-means finds groups but cannot name them. We would look at a group and <i>guess</i> "DSA Warrior", with no way to check the guess.
Skill level + persona combined	Rejected	Carries both sets of problems.
k-means on session style (this plan's 2026-09-16 version)	Kept, in a smaller role	Sound, but session labels alone give users little. It becomes the discovery step in Phase 4.

Skill level is replaced by **"better than your own usual"**, which already exists (the ▲/▼ against each person's 90-day baseline on the Insights page). It is honest and it judges nobody.

5. The design: three layers

Layer	Question it answers	Built with
1. Work type	What kind of coding was this session?	Rules first (Phase 1), then an ML model (Phase 3)
2. Persona	What kind of coder is this person lately?	A simple rule over layer 1
3. Group titles	Who led each kind of coding in the group this week?	A simple rule over layer 1

Only layer 1 uses machine learning, because it is the only question the rules genuinely cannot answer well (section 7). Layers 2 and 3 stay rules on purpose: anyone can check them by hand.

6. The five work types

Work type	What it looks like
DSA practice	Standalone problems: LeetCode, Codeforces, lab programs. One or two small files, compile-and-run loops, no Git.
Project building	Building an app or feature: many files, commits, package installs, builds and tests, code that stays.
Debugging	Fixing something broken in a project: debug sessions, failing builds or tests, repeated failed commands, little new code.
Learning & exploring	Reading code or docs, following a tutorial: mostly reading, many file switches, little writing.
Setup & config	Installing and configuring: package managers, Docker, config files, almost no code.

Two outcomes are not work types:

- **Not enough activity** — under 5 active minutes, or no edits and no commands. Nothing honest can be said. (Today's page calls this "Too short to classify", which is misleading for a 9-minute session with no edits; it gets renamed.)

- **Not sure** — the model's best guess is below 50%. The app asks the person instead of guessing.

7. Why the rules cannot do layer 1 well

- **No single signal marks a DSA session.** It is a combination: one file per problem, `g++` or `python` runs, no commits, file names like `two_sum.cpp`, a folder called `leetcode`.
- **Today's rules get it actively wrong.** The command classifier counts `g++` as a *build*, so a student fighting compile errors on a LeetCode problem trips the "debug grind" rule (two or more failed builds and low output).
- **Hand-picked thresholds do not travel.** Cut-offs such as "under 4 lines per minute" were chosen by hand, not learned from anyone's data, and people type at very different speeds.

8. The attributes

Every attribute below is derived from data the extension **already sends**. Each session becomes **18 numbers**.

Group	Attributes	Mainly separates
How you write (6)	<code>linesPerMin</code> , <code>churnRatio</code> , <code>readRatio</code> , <code>pasteShare</code> (share of inserted characters that arrived in large chunks: pastes, snippets, AI completions), <code>switchesPerMin</code> , <code>filesTouched</code> (capped at 10)	learning vs building; DSA (few files) vs projects (many)
What you run (6)	<code>commandsPerMin</code> , <code>runShare</code> (gcc / java / python commands), <code>packageShare</code> (npm / pip / docker), <code>commandFailRatio</code> , <code>buildFailRatio</code> , <code>testShare</code>	DSA vs setup vs projects
Debug, Git, focus (3)	<code>debugPerHour</code> , <code>commitsPerHour</code> , <code>deepShare</code>	debugging; projects (commit) vs DSA (rarely)
File-name hints (3)	<code>problemFileShare</code> (files named like problems), <code>configFileShare</code> (<code>package.json</code> , <code>Dockerfile</code> , <code>.yaml</code> , <code>.env</code> ...), <code>practiceFolder</code> (0/1: project folder named like <code>leetcode</code> , <code>dsa</code> , <code>cp</code> ...)	DSA and setup

Deliberately left out

Left out	Why
Session length, total lines	They measure how <i>big</i> a session was, not what <i>kind</i> ; the model would learn "short means DSA"
Raw file and project names	Privacy, and the model would memorise names instead of learning patterns
Time of day	Says <i>when</i> you coded, not what
Language	C++ is used for both DSA and projects; the model could learn a stereotype. Tested as an add-on later, with a per-language accuracy check.
Who the person is	The model must work for people it has never seen

Data limits to respect: file names are basenames of the file active at each upload, so `filesTouched` can undercount; commands are only seen in VS Code's own terminal; sessions recorded before the detailed trackers existed lack most of these signals and fall under *Not enough activity*.

9. Where the right answers (labels) come from

A model learns from examples with the right answer attached. Three sources:

1. **One tap per session.** The "How you worked" list shows the current guess with ✓ and **Change**. Correcting a guess takes a second.
2. **Blind questions.** On a random **20%** of prompts the app shows **no guess** and asks "What was this session?". Without these, people tend to click ✓ on whatever the rules said, and the model would learn to copy the rules — which would also make "beats the rules" untestable.
3. **Self-labelling cases.** A session inside a folder named `leetcode` is almost certainly DSA. These **weak labels** may help training, but they are **never used to measure accuracy**, and `practiceFolder` is removed from any model trained on labels derived from it.

Every label is saved with a **copy of the session's 18 numbers**, so training never has to re-derive old sessions. (The 2026-09-16 check found that feature vectors are currently thrown away after every page load; saving sessions fixes that.)

10. The model

First choice	Multinomial logistic regression : each attribute adds or removes points for each work type; the points become percentages. Easy to explain ("mostly because your files looked like practice problems"), and small enough to serve in JavaScript.
Compared against	A small decision tree (depth ≤ 4 , readable as if-then rules). Gradient boosting only if it is clearly better. The simplest model within 2 points of the best one wins.
Uneven classes	Setup sessions will be rarer than project sessions, so classes are weighted to be treated equally.
Scaling	Each attribute is standardised with the training set's mean and spread, saved with the model.
"Not sure"	Best probability under 50% → <i>Not sure</i> , and the app asks the person. Those answers are the most useful ones to learn from.
Serving	The trained model is a small JSON file in the repo (<code>backend/ml/workTypeModel.json</code> : attribute order, means, spreads, weights, version, training date and scores). JavaScript computes the prediction. No Python in production, no new server, and git history is the version history.

11. How we will know it works

1. **Test on people the model never saw.** Split by person, never by session: train on some people, test on others (5 rounds). Otherwise the model can pass by memorising one person's habits.
2. **Measure per work type**, in plain words:
 - *Precision* — when it says DSA, how often is it right?
 - *Recall* — of all real DSA sessions, how many did it catch?
 - *F1* — one number balancing the two; *macro-F1* averages it over the five types.
 - A *confusion table* shows which types it mixes up.
3. **Beat the rules.** The rules are scored on the same held-out, **blind** labels. The model ships only if its macro-F1 is at least **0.05 higher** and **no single work type's recall is more than 0.05 worse** than

the rules'. (These margins are this plan's proposal, not derived numbers.)

4. **Fairness checks.** Accuracy broken down per person and per language, so one heavy user or one language cannot hide a failure elsewhere.

If it fails, the rules stay. That is a valid result, and it is written up.

12. Layers 2 and 3: personas and group titles (no ML)

Persona. Over the last 30 days, using the person's own label where one exists, else the prediction:

Condition	Persona
Fewer than 5 classified sessions, or under 2 hours	<i>Warming up</i>
DSA practice $\geq$ 50% of classified minutes	DSA Warrior
Project building $\geq$ 50%	Project Grinder
Debugging $\geq$ 50%	Debugging King
Learning & exploring $\geq$ 50%	Code Explorer
Setup & config $\geq$ 50%	Setup Wizard
No type reaches 50%	All-rounder

It is recalculated once a week so it does not flicker, and always shown with its numbers ("62% of your coding minutes this month were DSA practice").

Group titles. Every Monday, for each work type, the member with the most minutes of that type in the previous week earns "*persona of group*". A title needs at least **60 minutes** of that type, and the group needs at least **3 members** active that week. Everyone can win something, which fixes a real problem: today both leaderboards rank by total hours, so only volume wins.

All personas and titles are positive, and anyone can hide theirs from groups.

13. What could go wrong

Problem	Fix
Too little data	Phases 1–2 ship without ML; training waits for the threshold in section 15
People click ✓ on every guess	Blind questions (section 9); accuracy measured on those only
Folder-name labels leak into the model	Weak labels never used for testing; <code>practiceFolder</code> dropped when training on them
Fake hours corrupt training and titles	Cap each 10-minute record at 10 minutes first (IMPROVEMENT_PLAN H-21)
One very active person dominates	Split by person; per-person accuracy check
Model learns "C++ means DSA"	Language excluded from v1; per-language accuracy check
A work type has too few examples	Types with under 30 confirmed sessions stay rule-based
Model drifts as new people join	Retrain monthly or every 200 new labels; each session records which model labelled it

14. What we will not claim

- **That it measures skill.** It describes what kind of work a session was.
- **Any accuracy figure before it is measured** on held-out people.
- **That a persona is a personality.** It is last month's mix of coding, nothing more.
- **That it is "AI".** It is logistic regression, one of the simplest and most explainable models.

15. Phases and effort

Phase	Work	Time
0. Fix first	Cap fake hours (H-21), stop showing member emails in groups (M-28), launch blockers (H-19, H-20), get 10+ people coding with extension 2.4.0	~3 h of code + the launch work already planned
1. Work types, no ML	Save sessions, 18 attributes, rules v1 with a DSA rule, one-tap and blind labels, Insights page changes	~25 h (+3 h optional extension change)
2. Personas & titles, no ML	Persona, weekly group titles, leaderboard and group tabs, privacy toggles, notifications	~23 h
3. Train the model	Export, train and evaluate in scikit-learn, JSON model, JavaScript inference with parity tests, "Not sure" prompts	~20 h
4. Discovery (optional)	k-means over the sessions the model is unsure about, looking for a missing work type	~6–8 h
Total		~77–82 hours, plus data collection

When to start Phase 3: at roughly **300 confirmed sessions from 10 or more people, with at least 30 per work type**. These are rules of thumb, not calculated numbers; the tests in section 11 are the real gate. Phases 1 and 2 are worth building even if the model never ships.

16. The k-means work kept from the first version of this plan

Phase 4 reuses the 2026-09-16 design, with its lessons intact:

- Cluster only **style** attributes, never size, or long sessions split from short ones.
- **Filter empty sessions first:** thousands of identical all-zero sessions form one perfectly tight group and make the silhouette score look good for the wrong reason.
- Choose k with the **elbow method** and the **silhouette score**, and match clusters to the previous run before reusing any names, because k-means numbers its groups randomly each time.

Its job changes from *labelling every session* to *finding a work type nobody thought of* — for example sessions that are mostly large pastes from AI tools.

17. Explaining it in an interview

"I only used ML where rules genuinely failed: deciding what kind of coding a session was — DSA, project, debugging, learning or setup — from 18 signals the extension already collects. I rejected skill levels because there's no way to validate them. Labels come from one-tap confirmations, with a share of blind questions so people don't just agree with the rules. The model is logistic regression served as JSON in JavaScript, tested on people it never saw, and it only replaces the rules if it beats them. Personas and group titles are simple rules on top. It isn't built yet — we have one active user — so the first two phases ship without ML and collect the labels."

Likely follow-up questions

- *Why not skill level?* No ground truth, a harmful label if wrong, and easy to game.
- *Why logistic regression?* Explainable per attribute, tiny to serve in JavaScript, and a strong baseline; a tree or boosting must beat it clearly to replace it.
- *How do you stop the model just copying your rules?* Blind questions, and accuracy is measured only on those.
- *Why split by person?* Otherwise it can score well by memorising one person's habits.
- *Where does k-means fit?* Discovering missing work types among uncertain sessions, not labelling.